Privacy Attacks against ML models and Defense Techniques

Lecture Goals

- Privacy risks in machine learning
- Membership inference
- Why should we care?
- Threat models considerations
- Attacks
- Defenses (next lesson)

Machine Learning on Sensitive Data

- **Medical:** predictive models for patient outcomes
  - E.g., diagnosis: disease prediction, treatment plans: personalised recommendations
  - Subject to:
    - HIPAA compliance: US
    - General Data Protection Regulation (GDPR): EU
      - Please note that it applies to all personal data processing in the EU, including healthcare
      - Health data is classified as “special category data”, which requires stronger protection and a valid legal basis (e.g., explicit consent, public health, medical care).
    - EU Artificial Intelligence Act (AI Act): EU
      - AI systems used in healthcare are typically classified as high-risk AI and must meet transparency, risk management, and governance requirements.

Machine Learning on Sensitive Data

- **Finance**: credit scoring
  - E.g., loan approval, fraud prediction, fraud detection, ...
  - Subject to:
    - Right to Financial Privacy Act (RFPA): US
    - GDPR: EU
      - Individuals have rights regarding decisions based solely on automated processing (like credit scoring).
      - Transparency requirements: Organizations must explain the logic and impact of automated decisions.
      - Data protection principles: fairness, purpose limitation, minimization, accuracy.

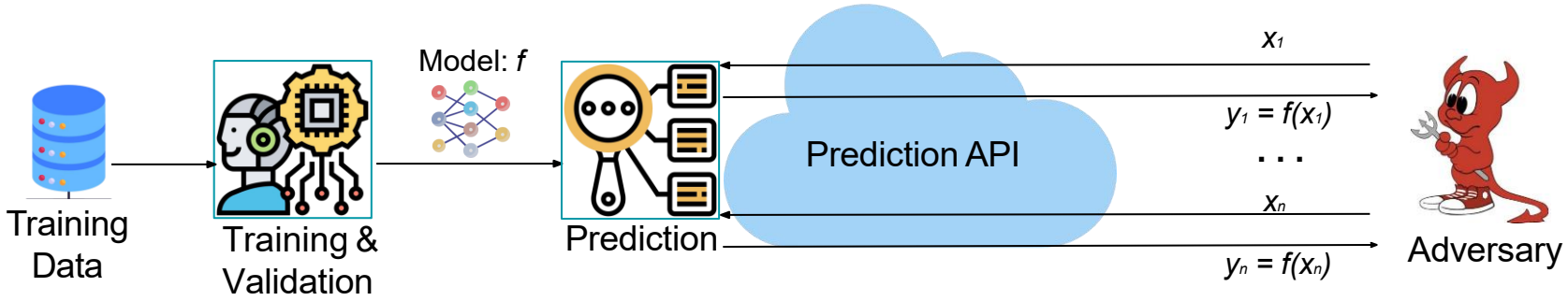
Machine Learning on Sensitive Data

- Education: adaptive learning platforms
  - E.g., predicting academic performance, personalized recommendations of materials
    - Family Educational Rights and Privacy Act (FERPA): US
    - GDPR: EU
      - Lawful basis for processing: legitimate interest, consent (especially for minors or optional services)
      - Protection of children's data: parental consent for certain online services stricter transparency requirements

Privacy Risks in Machine Learning

Membership Inference	● From f(x) =y, determine whether or not x is in training set
Model Inversion	● From output, infer sensitive info (e.g., face)
Training Data Extraction	● Extract some # training samples (e.g., 1% training samples)
Attribute Inference	● From output, infer sensitive attribute (e.g., race)
Model Extraction	● From public data, train f' ~f

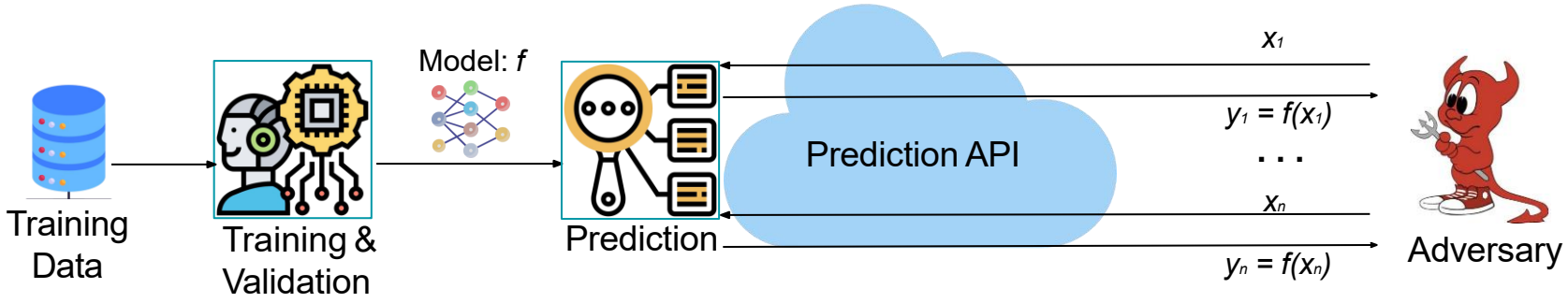
Privacy: Membership Inference Attack



Adversarial Goal: inspect a model's predictions to determine whether or not a sample was used to train it.

Shokri *et al*. https://arxiv.org/abs/1610.05820; Choquette-Choo *et al.* https://arxiv.org/abs/2007.14321v2

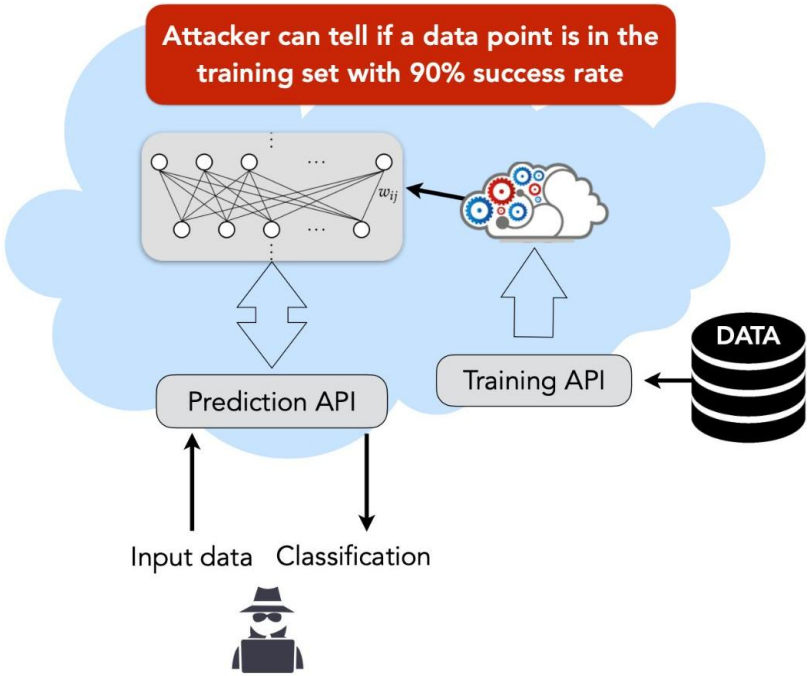
Privacy: Model Inversion Attack



Adversarial Goal: given a model's prediction (e.g., with confidence scores), reconstruct a sensitive training data-point (e.g., a person's face, a medical record)

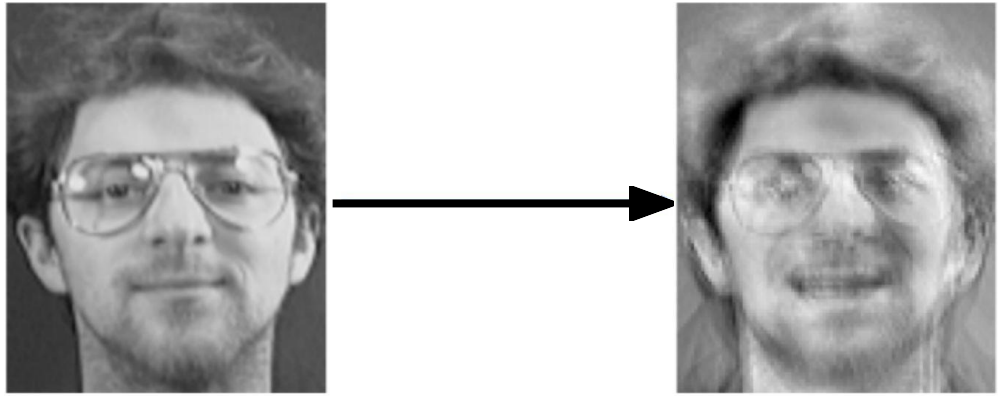
Fredrikson *et al.*, 2015. Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures

Privacy: Membership Inference …



Shokri *et al*. https://arxiv.org/abs/1610.05820

Privacy: Model Inversion Attack



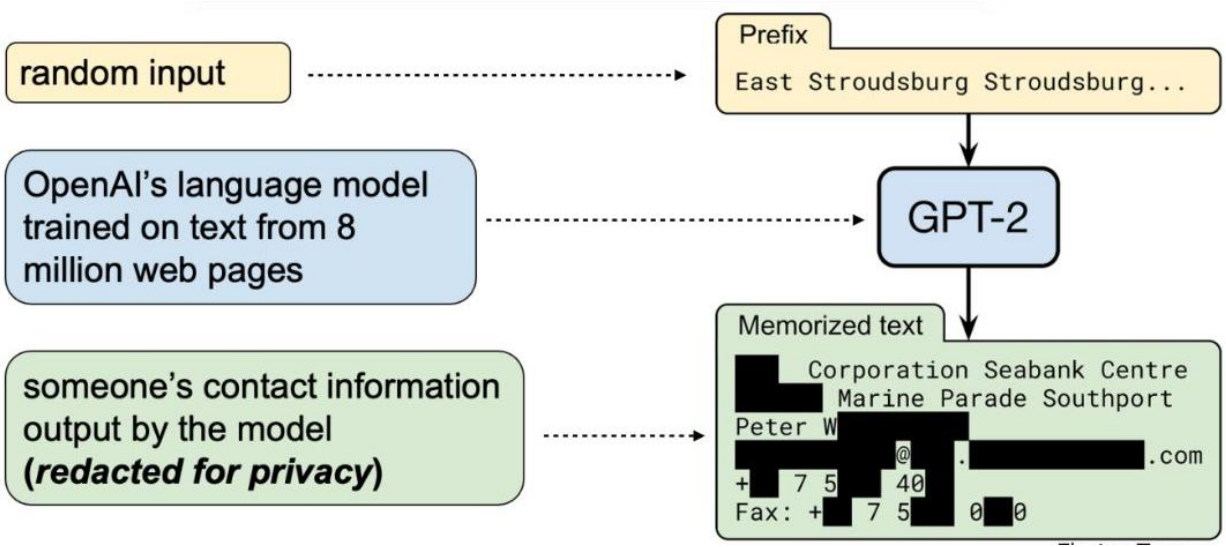
Original face from training data

Reconstructed face after inversion attack

"The attacker is given only the person's name and access to a facial recognition system that returns a class confidence score"

Fredrikson *et al.*, 2015. Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures

Privacy: Training Data Extraction from LLMs



"Given query access to a neural network language model, we extract an individual person's name, email address, phone number, fax number, and physical address. The example in this figure shows information that is all accurate so we redact it to protect privacy."

Carlini *et al.*, 2021. Extracting Training Data From Large Language Models

Privacy: Training Data Extraction From Diffusion Models

Training Set

Caption: Living in the light with Ann Graham Lotz

Generated Image

Prompt: Ann Graham Lotz

- "Diffusion models memorize individual training examples and generate them at test time.
- **Left**: an image from Stable Diffusion's training set (licensed CC BY-SA 3.0).
- **Right**: a Stable Diffusion generation when prompted with "Ann Graham Lotz". The reconstruction is nearly identical"

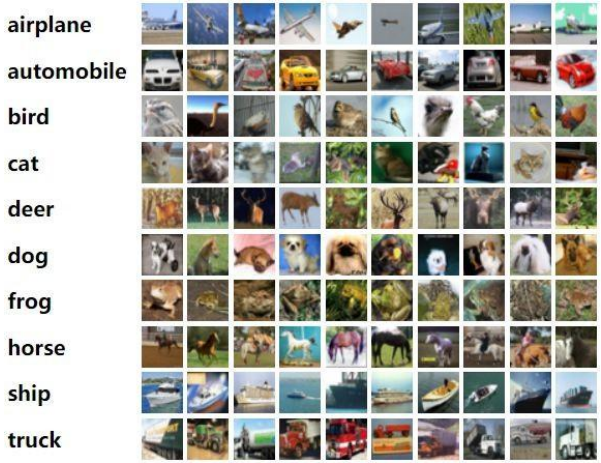
Original:

Generated:

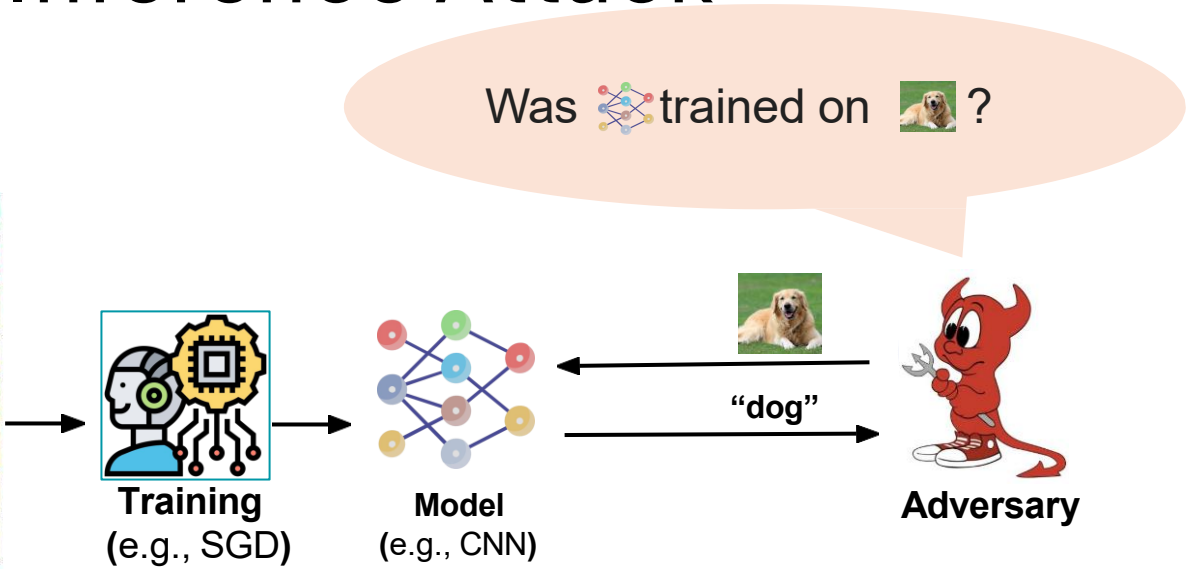
Carlini *et al.*, 2023. Extracting Training Data From Diffusion Models

Membership Inference Attacks

Membership Inference Attack



Training Data (e.g., CIFAR-10)



Adversarial Goal: inspect a model's predictions to determine whether or not a sample was used to train it.

Shokri *et al*. https://arxiv.org/abs/1610.05820

Membership Inference: A Bit Formally

A MIA is a statistical attack where an adversary aims to infer whether $x \in \mathcal{D}^{train}$. Given a candidate sample x, a model f trained on $\mathcal{D}^{train}$, and adversary's knowledge (about f and $\mathcal{D}^{train}$) denoted by $\mathcal{K}$, the goal of MIA is to determine whether x is used to train f. More formally, MIA is defined as an attack function $\mathcal{A}$ as: $\mathcal{A}(x, f, \mathcal{K}) \rightarrow \{0, 1\}$, where 0 means $x \notin \mathcal{D}^{train}$ and 1 means $x \in \mathcal{D}^{train}$.

Why Care About Membership Inference?

- **Curiosity** (e.g., did hospital x use my data?)
- **Gather intelligence** for future attacks
- As **building block** for data extraction attacks
- **Auditing** correctness of privacy mechanisms

If private data can be inferred given access to a model, then it is not safe to share the model!

Why is Membership Inference Possible?

Confidence (| is from 's training set)

>

Confidence (| is not from 's training set)

- **Low-hanging fruit:**
  - Statistical distinguishability of model's confidence on members vs. non-members
  - Root cause: **overfitting**: models are more confident on members of their training set than on non-members
- Overfitting is not the only cause

Threat Model Considerations: Adversary

- **We will focus on black box access to classifiers**
- **Probability-based**:
  - Has access to some set of samples from same distribution as training data
  - Oracle access to prediction API with label probability scores and predicted label
  - Model architecture (e.g., Machine Learning as Servivce provider published model details)
- **Label Only**:
  - Oracle access with only label access
  - Can do augmentation and/or perturbation of target inputs to observe model's sensitivity

Threat Model Considerations: Defender

- Machine Learning as a Service (MLaaS) API exposes only predictions, not intermediate activations
- Adversary can’t compute gradients through the model
- MLaas API is rate-limited: e.g., adversary can’t make more than 5K queries/hour
- Adversary can’t perform more than $50K worth of compute (limits model extraction adversaries)

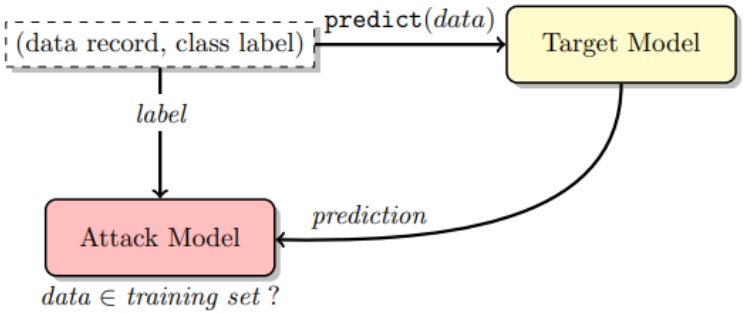
The Shadow Model Attack

- **Goal**: training a model to infer whether or not a data point is in the training set of the target model.
- **Assumptions**:
  - attacker has access to a shadow dataset *that comes from same distribution as the training set*
    - One can also generate some (see section V-C of the paper)
  - attacker has probability distributions of each prediction

Shokri *et al*. https://arxiv.org/abs/1610.05820

The Shadow Model Attack: overview

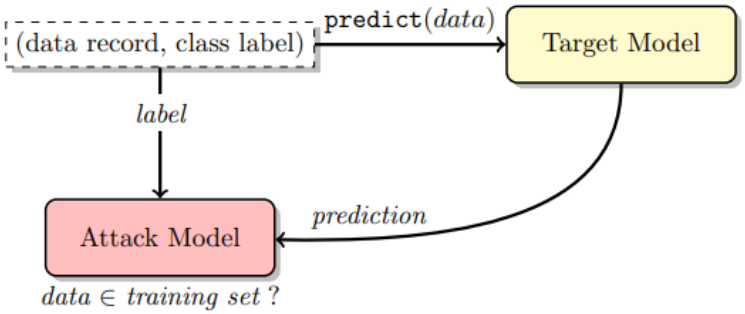
- Target model f_target is a classifier on c_target classes
- Let A be the attack model.
  - Input x_attack = correctly labeled record + prediction vector of size c_target.
  - Goal of the attack = membership inference → A is a binary classifier with two output classes, “in” and “out.”



Shokri *et al*. https://arxiv.org/abs/1610.05820

The Shadow Model Attack: overview

- For a labeled record (x, t)
- Use the target model to compute the prediction vector **y** = f_target(x).
- The distribution of **y** (classification confidence values) depends heavily on the true class of x. This is why we pass the true label t of x in addition to the model's prediction vector **y** to the attack model.
- Given how the probabilities in **y** are distributed around t, the attack model computes the membership probability:
- Pr{(x, t) ∈ D train _target} =
- probability that ((x, t), **y**) belongs to the "in" class or, equivalently, that x is in the training dataset of f_target().



Shokri *et al*. https://arxiv.org/abs/1610.05820

The Shadow Model Attack

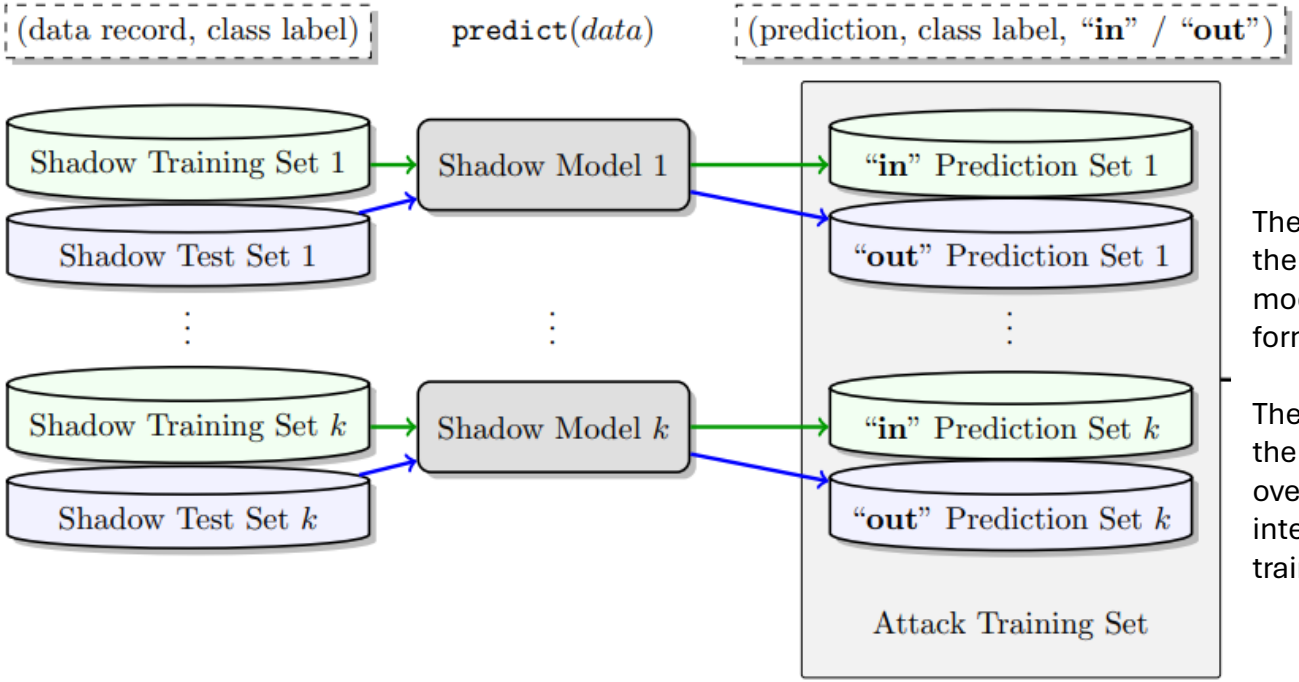
- Step 1: Training data collection
- Step 2: Attack model training
- Step 3: Perform attack

Shokri *et al*. https://arxiv.org/abs/1610.05820

Step 1: Training Data Collection

- **Assumptions**: Attacker has access to $\boldsymbol{D}_{shadow}$ (same distribution as $\mathbf{D}_{train}$)
- **Goal:** Train shadow models on $\boldsymbol{D}_{shadow}$ to produce training data for the attack model *A*
- **Steps**:
  - Partition $\boldsymbol{D}_{shadow}$ into $\boldsymbol{D}_{in}$ and $\boldsymbol{D}_{out}$
  - **Choose a model architecture** and **train** the shadow $\boldsymbol{f}_{shadow}$ on $\boldsymbol{D}_{in}$
  - For each $(\boldsymbol{x},\boldsymbol{t}) \in \boldsymbol{D}_{in}$
    - compute $\boldsymbol{y} = f_{shadow}(\boldsymbol{x})$
    - create A's "in" training data-point **[t, y, in]**
  - For each $(\boldsymbol{x},\boldsymbol{t}) \in \boldsymbol{D}_{out}$
    - compute $\boldsymbol{y} = f_{shadow}(\boldsymbol{x})$
    - create ***A***'s "out" training data-point **[*v, t, out*]**

Step 1: Training Data Collection



The training datasets of the target and shadow models have the same format but are disjoint.

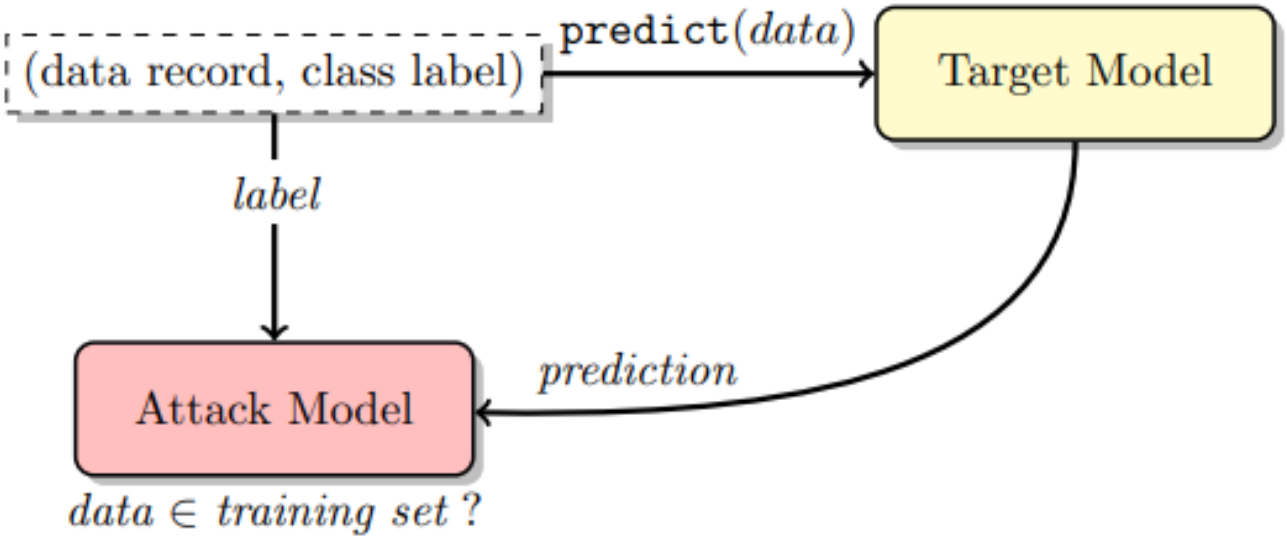
The training datasets of the shadow models may overlap. All models' internal parameters are trained independently.

Step 2: Attack Model Training

For each class v in [1,…, c_target], prepare a dataset from the Attack Training Set

$D_{in\text{-}out}$ = [v, **y**, *in*] U [v, **y**, *out*]

Step 3: Performing the Attack



Results - Metrics

- We evaluate the attack using the standard precision and recall metrics.
  - **Precision** is the fraction of the records inferred as members of the training dataset that are indeed members.
  - **Recall** measures coverage of the attack, i.e., the fraction of the training records that the attacker can correctly infer as members.
- Most measurements are reported per class because the accuracy of the attack can vary considerably for *different classes*. This is due to the difference in size and composition of the training data belonging to each class and highly depends on the dataset.

Results (CIFAR)

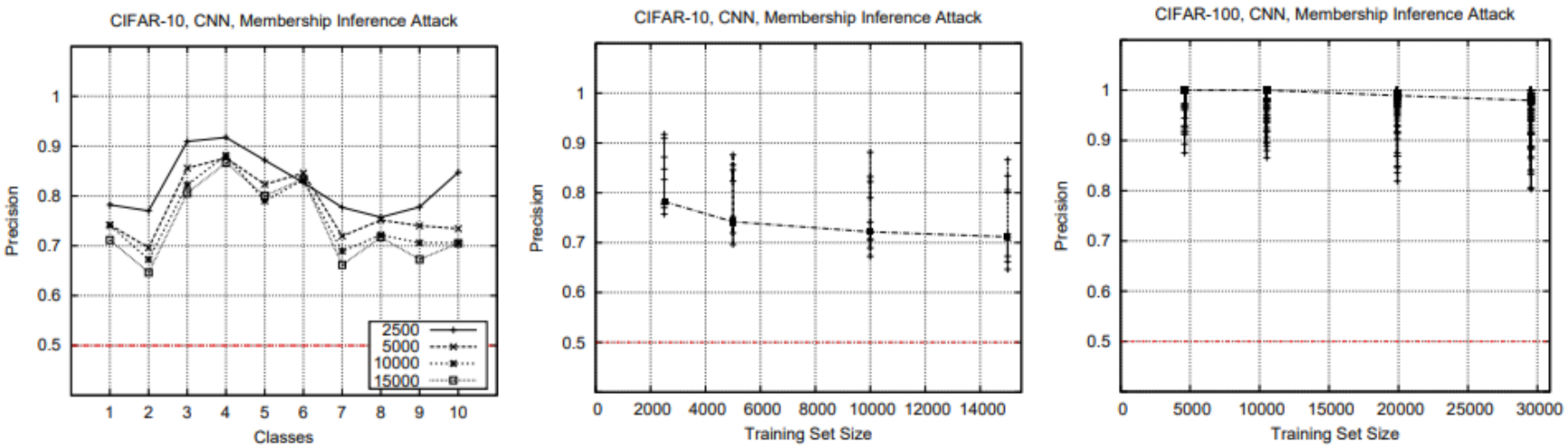


Fig. 4: Precision of the membership inference attack against neural networks trained on CIFAR datasets. The graphs show precision for different classes while varying the size of the training datasets. The median values are connected across different training set sizes. The median precision (from the smallest dataset size to largest) is $0.78, 0.74, 0.72, 0.71$ for CIFAR-10 and $1, 1, 0.98, 0.97$ for CIFAR-100. Recall is almost 1 for both datasets. The figure on the left shows the per-class precision (for CIFAR-10). Random guessing accuracy is 0.5.

Number of classes affects the results

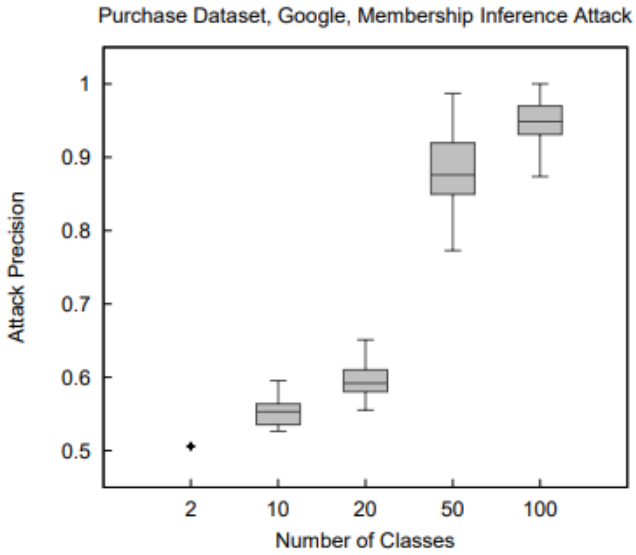


Fig. 10: Precision of the membership inference attack against different purchase classification models trained on the Google platform. The boxplots show the distribution of precision over different classification tasks (with a different number of classes).

Why does the attack work?

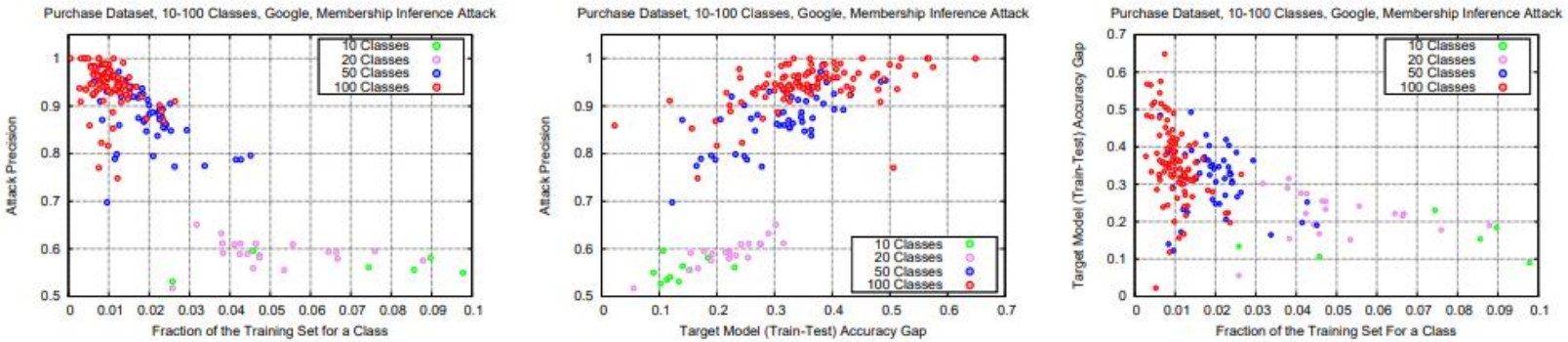


Fig. 11: Relationship between the precision of the membership inference attack on a class and the (train-test) accuracy gap of the target model, as well as the fraction of the training dataset that belongs to this class. Each point represent the values for one class. The (train-test) accuracy gap is a metric for generalization error [18] and an indicator of how overfitted the target model is.

Success of membership inference is directly related to the

1. generalizability of the target model and
2. diversity of its training data

If the model overfits and does not generalize well to inputs beyond its training data, or if the training data is not representative, the model leaks information about its training inputs.

Hands-on

- https://github.com/elenasofia98/Artificial-Intelligence-and-Cybersecurity/blob/main/MIA_Shokri_2016.ipynb

Limitations of original MIA definition

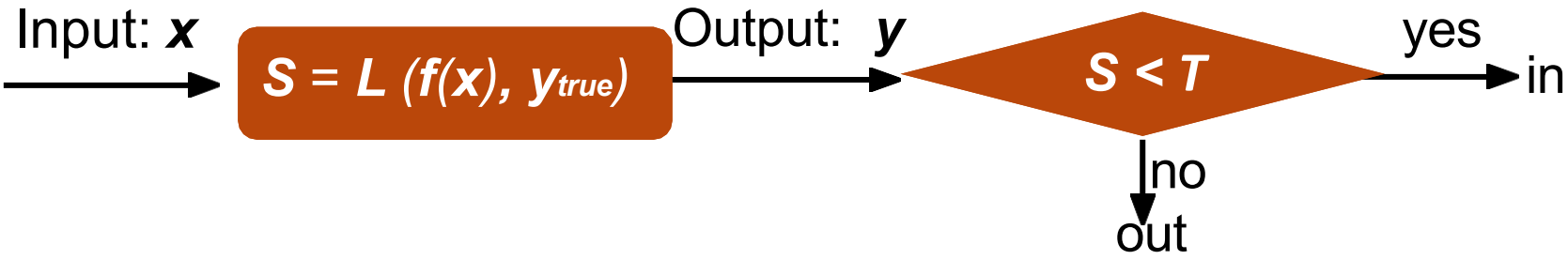
- The original formulation of MIA can be used as long as it is feasible for the attacker to construct **shadow models that are similar to the target model**.
- Training shadow models may be too expensive!
  - Training (<u>a large number of</u>) shadow models for target models with billions of parameters is simply too costly
- For this reason, the formulation for MIA has been adapted, generally eliminating shadow model developments

Threshold-based MIA

- This adversary works without any shadow model, i.e., the attack only relies on the outcomes obtained from the target model when querying it with target data points.
- Compute some measures over the observed output
- Establish a threshold over the computed quantity
- Use the threshold to establish if a data point is a member or not

Prediction Loss-based Attack

- Assign a score to a data-point ***x*** using the target model's loss as $S = L\,(f(x), y_{true})$ for that particular data-point.
- ***S*** can be converted into a binary label in/out by thresholding at some chosen threshold ***T***, to output "in" if the loss is below **T**
- **T:** can be estimated as the average of maximum loss of ***f*** on training data (which could be public information when reported by the model owner)



Intuition: models are trained to minimize loss and can achieve zero loss for training data.

Yeom *et. al.*, https://arxiv.org/pdf/1709.01604.pdf

Prediction Loss-based Attack

	Our work	Shokri et al. [7]
Attack complexity	Makes only one query to the model	Must train hundreds of shadow models
Required knowledge	Average training loss L_S	Ability to train shadow models, e.g., input distribution and type of model
Precision	0.505 (MNIST) 0.694 (CIFAR-10) 0.874 (CIFAR-100)	0.517 (MNIST) 0.72-0.74 (CIFAR-10) > 0.99 (CIFAR-100)
Recall	> 0.99	> 0.99

Yeom *et. al.*, https://arxiv.org/pdf/1709.01604.pdf

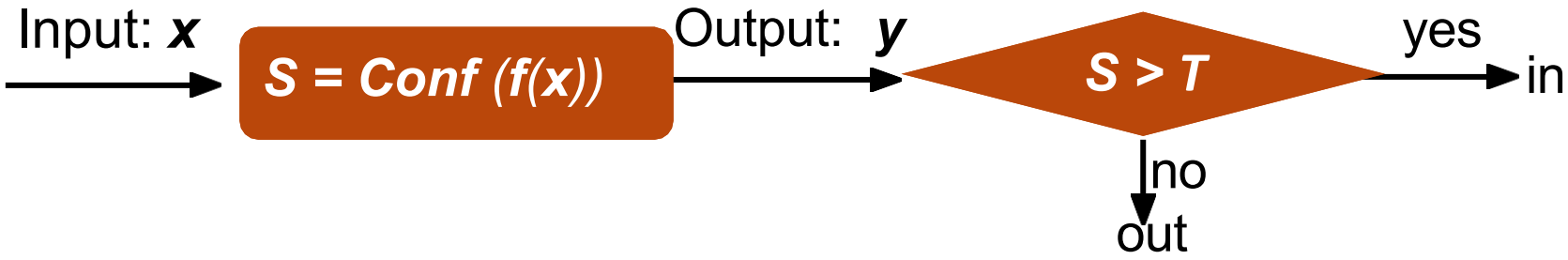
Why is this simple attack successful?

- Ther is a dependence of membership attack success on **generalization error**
  - Discussion will follow
- Experiments on real data confirm that this connection matters in practice
- Despite requiring significantly less computation and adversarial background knowledge, this attack performs almost as well as the shadow model attack (on tested datasets).

Yeom *et. al.*, https://arxiv.org/pdf/1709.01604.pdf

Prediction Confidence-based Attack

- Assign a score to a data-point ***x*** based on the model's confidence in its predicted class
- A confidence threshold ***T*** can be chosen based on the average/minimum on training data if available or can be determined on observations from a batch of predictions

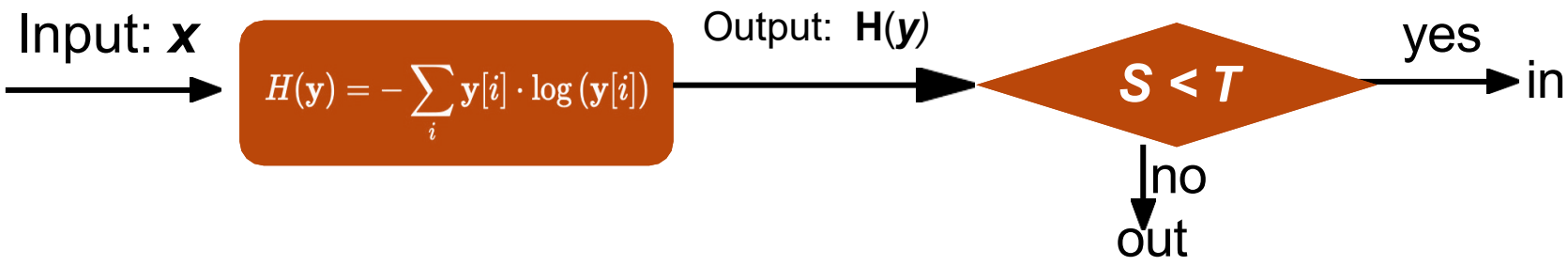


Intuition: models are often more confident on training examples (even when predictions are incorrect)

Salem *et. al.*, https://arxiv.org/abs/1806.01246

Prediction Entropy-based Attack

- Assign a score to a data-point ***x*** using the entropy of the probability distribution of classes as: ***H (f(x) = y[1] …y[k])*** for ***k*** # of classes/labels
- **T:** see next slide



Intuition: models are often more confident on training examples.

Salem *et. al.*, https://arxiv.org/abs/1806.01246

Prediction Entropy-based Attack

- How to choose a threshold **T**?
- Generate a sample of random points in the feature space of the target data point.
  - For image datasets including CIFAR-10, CIFAR-100, MNIST, and Face, generate random images, where the value of each pixel is drawn from a uniform distribution.
  - For datasets with binary features generate 0 and 1 for each feature according to an unbiased coin flip.
  - *They did not test this procedure on text or categorical features "One way to tackle this is to collect News articles or people's records (with the same features as in the Adult dataset) from the Internet as the random points"*
- Hypothesize that these points act as the nonmember points.
- **Compute the statistic (entropy) for this points a compute the top t percentile, this will be the threshold.**
- Empirically that there exists a choice of t percentile that works well and generalizes across all the dataset
  - They found 10 (see Figure)

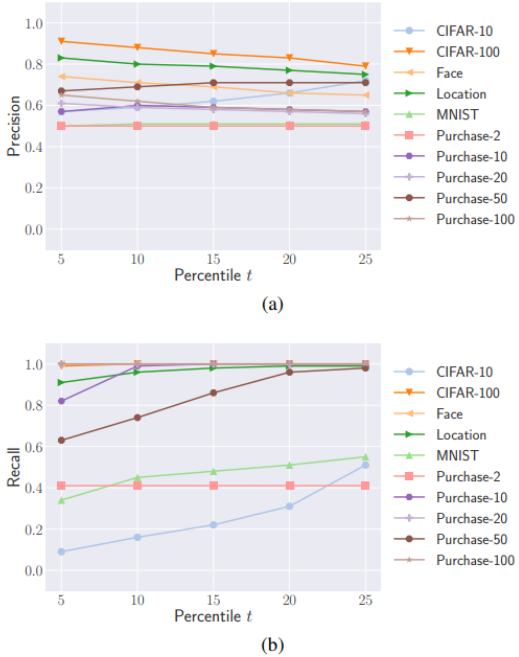


Fig. 10: The relation between the percentile of the maximal posterior, i.e., threshold, (x-axis) and the third adversary's performance (y-axis). (a) precision, (b) recall.

Salem *et. al.*, https://arxiv.org/abs/1806.01246

Prediction Entropy-based Attack

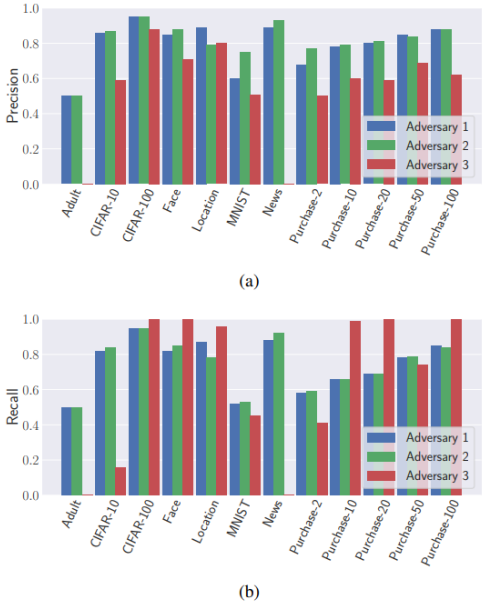


Fig. 12: Comparison of the three different adversaries' performance. (a) precision, (b) recall.

Salem *et. al.*, https://arxiv.org/abs/1806.01246

Limitations

- The computation of the threshold is one key aspect
- A static, absolute threshold does not control for the intrinsic complexity of each utterance:
  - It is better to calibrate the threshold for each utterance
  - To account for this, the idea of difficulty calibration mechanisms has been introduced:
    - It aims to quantify the intrinsic complexity of a data sample
    - Subsequently use this value to regularize model scores before comparing them to a threshold value

Likelihood Ratio Attacks (LiRA)

- Measure the difficulty of a target point by feeding it to reference models
- This help provide a perspective into how likely that target point is in the given domain
- But first we discuss this attack, let's revise our metrics (again)

Carlini *et. al.*, https://arxiv.org/pdf/2112.03570

Metrics (again!)

- Membership inference attacks should be evaluated by considering their **true-positive rate (TPR) at low false-positive rates (FPR) (TPR@low-FPR)**.
- This objective of designing methods around low false-positive rates is typical in many areas of computer security.

Carlini *et. al.*, https://arxiv.org/pdf/2112.03570

Metrics (again!)

		Inference results	
		Positive sample	Negative sample
Labels	Positive sample	TP	FN
	Negative sample	FP	TN

https://www.sciencedirect.com/topics/computer-science/true-positive-rate

- The true positive rate (TPR) gives the proportion of actual positives which are correctly identified as such. (Higher is better)
- The false positive rate (FPR) gives the proportion of falsely identified positives amongst all actual negatives. (lower is better)

Metric	Formula
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$
Precision	$\frac{TP}{TP+FP}$
Recall	$\frac{TP}{TP+FN}$
True Positive Rate (TPR)	$\frac{TP}{TP+FN}$
Specificity	$\frac{TN}{TN+FP}$
F1 Score	$\frac{2TP}{2TP+FP+FN}$
False Positive Rate (FPR)	$\frac{FP}{FP+TN}$

Table 1: Common classification performance metrics

TPR@low-FPR

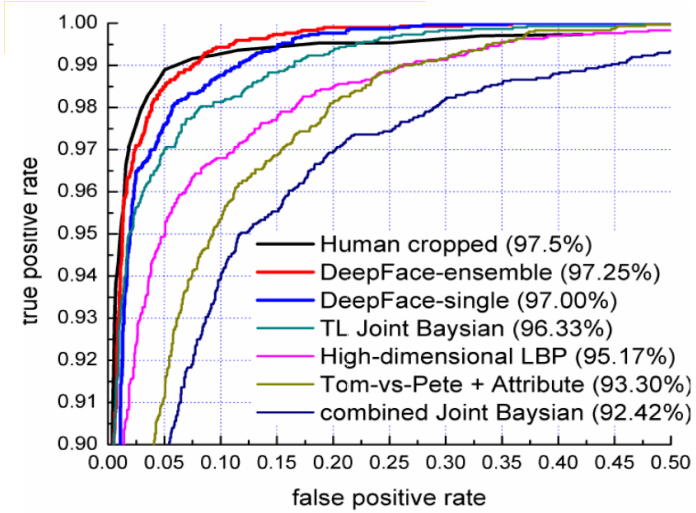
- It is essential for applications where **low false alarm rates are required** while **maximizing the identification of true positives**.
- Examples of Use Cases:
  - Medical Diagnostics: High TPR is prioritized to detect diseases, but a low FPR is needed to prevent unnecessary treatment or anxiety.
  - Cybersecurity/Fraud Detection: The goal is to detect attacks (high TPR) without blocking legitimate user activity (low FPR).
  - Machine Learning Security: Evaluating how well an attacker can identify training data, where a high TPR at low FPR indicates a significant leak

Metric	Formula
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$
Precision	$\frac{TP}{TP+FP}$
Recall	$\frac{TP}{TP+FN}$
True Positive Rate (TPR)	$\frac{TP}{TP+FN}$
Specificity	$\frac{TN}{TN+FP}$
F1 Score	$\frac{2TP}{2TP+FP+FN}$
False Positive Rate (FPR)	$\frac{FP}{FP+TN}$

Table 1: Common classification performance metrics

TPR@low-FPR

- This metric focuses on the top-left corner of the Receiver Operating Characteristic (ROC) curve.
  - ROC curve, compares the TPR and FPR for all possible choices of the decision threshold τ.
- The ideal form of the curve would be a vertical step function of height 1 at false-positive rate = 0. The worst possible curve would be the inverse of this.



- Applying this to the data in the figure:
  - “DeepFace Ensemble” dominates all other approaches except human-cropped
  - “Human cropped” appears to be the best choice if very low fase positives are needed
  - “Human cropped” is inferior for higher allowable false positives

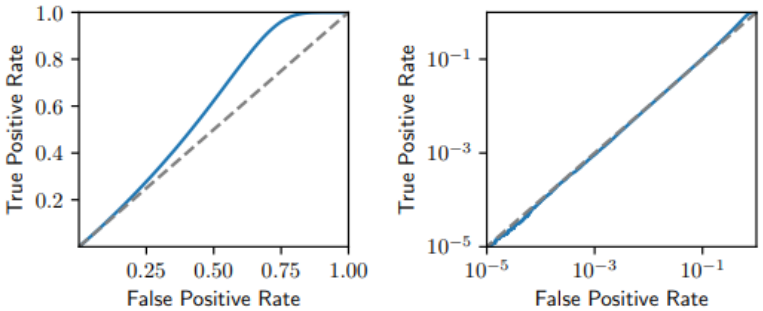
TPR@low-FPR for privacy protection

- If a membership inference attack can **reliably** violate the privacy **of even just a few users in a sensitive dataset**, it has succeeded.
- Conversely, an attack that only unreliably achieves high aggregate attack success rate should not be considered successful.
- This entails that aggregate metrics are not sufficient
- We should thus consider metrics that emphasize positive predictions (i.e., membership guesses) over negative (non-membership) predictions.
- A natural choice is to consider the tradeoff between the true-positive rate (TPR) and false-positive rate (FPR).
  - Intuitively, an attack should maximize the true-positive rate (many members are identified), while incurring few false-positives (incorrect membership guesses).
  - We prefer this to a precision/recall analysis because TPR/FPR is independent of the (often unknown) prevalence of members in the population.

Carlini *et. al.*, https://arxiv.org/pdf/2112.03570

Revisiting the results for the LOSS attack

- From now on the threshold-based attacks that uses the loss as statistic will be referred to as the LOSS attack
- Trained a CIFAR-10 model, and obtained a LOSS attack's balanced accuracy is 60%.
  - This is (much) better than random guessing (50%), and so one might reasonably conclude that the attack is useful and practically worrying
- However, this attack completely fails at <u>confidently</u> identifying any members!
  - Figure 2 reports the ROC curve for the LOSS attack. The attack fails to achieve a TPR better than random chance at any FPR below 20% —it is therefore ineffective at <u>confidently</u> breaching the privacy of its members.



(a) linear scale (b) log scale

Fig. 2: ROC curve for the LOSS baseline membership inference attack, shown with both linear scaling (left), also and log-log scaling (right) to emphasize the low-FPR regime.

Carlini *et. al.*, https://arxiv.org/pdf/2112.03570

Likelihood Ratio Attacks (LiRA)

- Let's go back to the LiRA attacks, and reformulate MIA a bit
- Given a model f and a data point x, a MIA requires the adversary to distinguish between two "worlds":
  - one where f is trained on a randomly sampled dataset that contains a target point x,
  - and one where f is not trained on x.
  - It is thus natural to see a membership inference attack as performing a hypothesis test to guess whether or not f was trained on x

$$H_0 : x \notin D_{\text{train}} \quad \text{(non-member)}$$
$$H_1 : x \in D_{\text{train}} \quad \text{(member)}$$

Carlini *et. al.*, https://arxiv.org/pdf/2112.03570

Likelihood Ratio Attacks (LiRA)

- Let s(x) denote the statistic observed by the attacker (e.g., the model's confidence or loss for x). The attacker models the distribution of this statistic under both hypotheses:

$$s(x) \sim P_{\text{in}}(s) \quad \text{if } x \in D_{\text{train}}$$
$$s(x) \sim P_{\text{out}}(s) \quad \text{if } x \notin D_{\text{train}}$$

- The optimal decision rule for distinguishing the two hypotheses is given by the likelihood ratio test. The likelihood ratio is

$$\Lambda(s) = \frac{P_{\text{in}}(s)}{P_{\text{out}}(s)}$$

Carlini *et. al.*, https://arxiv.org/pdf/2112.03570

Likelihood Ratio Attacks (LiRA)

- The attacker predicts that x is a member if $\Lambda(s(x)) > \tau,$ where tau is a threshold chosen to achieve a desired false-positive rate.
- This can also be written in log-form

$$\log \Lambda(s(x)) = \log P_{\text{in}}(s(x)) - \log P_{\text{out}}(s(x)).$$

- Because the attacker does not know the true distributions P_in and P_out, they estimate them using **shadow models**.
  - Models are trained on datasets drawn from the same distribution as the target training data.
  - For each shadow model the attacker records the statistic s(x) for both training examples (members) and non-training examples (non-members)

Carlini *et. al.*, https://arxiv.org/pdf/2112.03570

Likelihood Ratio Attacks (LiRA)

- In practice, the attack assumes that the statistics follow Gaussian distributions:

$$P_{\text{in}}(s) = \mathcal{N}(\mu_{\text{in}}, \sigma_{\text{in}}^2)$$
$$P_{\text{out}}(s) = \mathcal{N}(\mu_{\text{out}}, \sigma_{\text{out}}^2)$$

- Using these models, the attacker computes the likelihood of the observed statistic under both hypotheses and evaluates the likelihood ratio.
- The final membership score is $\text{score}(x) = \log P_{\text{in}}(s(x)) - \log P_{\text{out}}(s(x)).$
- The example is predicted to be a member if $\text{score}(x) > \tau.$
- By adjusting the threshold τ , the attacker can control the trade-off between the true-positive rate (TPR) and the false-positive rate (FPR).

Carlini *et. al.*, https://arxiv.org/pdf/2112.03570

Memorization and per-example hardness

- By casting membership inference as a Likelihood-ratio test, it becomes clear why the LOSS attack (and those that build on it) are ineffective:
  - By directly thresholding the quantity loss(f(x), y), this attack implicitly assumes that:
    - the losses of all examples are a priori on an equal scale,
    - and that the inclusion or exclusion of one example will have a similar effect on the model as any other example.
  - No other property of the example is taken into account

Carlini *et. al.*, https://arxiv.org/pdf/2112.03570

Memorization and per-example hardness

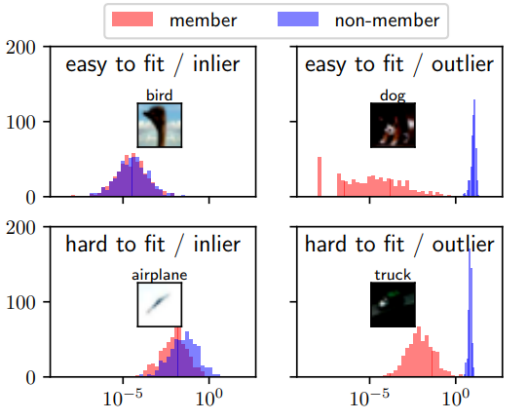


Fig. 3: Some examples are easier to fit than others, and some have a larger separability between their losses when being a member of the training set or not. We train 1024 models on random subsets of CIFAR-10 and plot the losses for four examples when the example is a member of the training set ($\tilde{\mathbb{Q}}_{\text{in}}(x, y)$, in red) or not ($\tilde{\mathbb{Q}}_{\text{out}}(x, y)$, in blue).

- Not all data are equal
  - Inlier have small effects on their loss when they are included in the training data
  - Outliers have cause larger effects
- The left column shows examples with low loss when omitted from the training set, those in the right column have high loss.
- On rows we compare how easy the examples are to fit. The examples in the top row have very low loss when trained on, while the examples in the bottom row have higher loss.
- This view illustrates the shortcomings of prior attacks (e.g., the LOSS attack): a global threshold on the observed loss cannot distinguish between the different scenarios

Carlini *et. al.*, https://arxiv.org/pdf/2112.03570

Memorization and per-example hardness

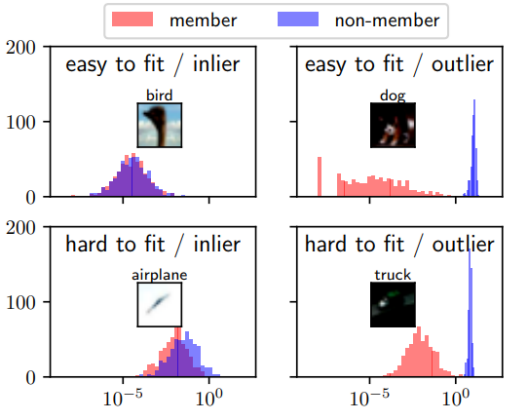


Fig. 3: Some examples are easier to fit than others, and some have a larger separability between their losses when being a member of the training set or not. We train 1024 models on random subsets of CIFAR-10 and plot the losses for four examples when the example is a member of the training set ($\tilde{\mathbb{Q}}_{\text{in}}(x, y)$, in red) or not ($\tilde{\mathbb{Q}}_{\text{out}}(x, y)$, in blue).

- The goal of a membership inference adversary is to distinguish the two distributions in Figure 3 for a given example.
- The Likelihood-ratio test we described considers the hardness of each example individually by modeling separate pairs of distributions for the loss P_in and P_out for each example x

Carlini *et. al.*, https://arxiv.org/pdf/2112.03570

Estimating the likelihood-ratio with parametric modeling

- N shadow models on random samples from the data distribution D
  - Half of these models are trained on the target point (x, y) → IN models
  - Half are not (we call these respectively IN and → OUT models
- We then fit two Gaussians to the confidences of the IN and OUT models on x.
- Query the confidence of the target model f on x and output a parametric Likelihood-ratio test.

Algorithm 1 Our online Likelihood Ratio Attack (LiRA). We train shadow models on datasets with and without the target example, estimate mean and variance of the loss distributions, and compute a likelihood ratio test. (In our **offline** variant, we omit lines 5, 6, 10, and 12, and instead return the prediction by estimating a single-tailed distribution, as is shown in Equation (4).)

Require: model f, example (x, y), data distribution $\mathbb{D}$

1: $\text{confs}_{\text{in}} = \{\}$
2: $\text{confs}_{\text{out}} = \{\}$
3: **for** N times **do**
4: $\quad D_{\text{attack}} \leftarrow^{\$} \mathbb{D}$ ▷ *Sample a shadow dataset*
5: $\quad f_{\text{in}} \leftarrow \mathcal{T}(D_{\text{attack}} \cup \{(x, y)\})$ ▷ *train IN model*
6: $\quad \text{confs}_{\text{in}} \leftarrow \text{confs}_{\text{in}} \cup \{\phi(f_{\text{in}}(x)_y)\}$
7: $\quad f_{\text{out}} \leftarrow \mathcal{T}(D_{\text{attack}} \backslash \{(x, y)\})$ ▷ *train OUT model*
8: $\quad \text{confs}_{\text{out}} \leftarrow \text{confs}_{\text{out}} \cup \{\phi(f_{\text{out}}(x)_y)\}$
9: **end for**
10: $\mu_{\text{in}} \leftarrow \texttt{mean}(\text{confs}_{\text{in}})$
11: $\mu_{\text{out}} \leftarrow \texttt{mean}(\text{confs}_{\text{out}})$
12: $\sigma^2_{\text{in}} \leftarrow \texttt{var}(\text{confs}_{\text{in}})$
13: $\sigma^2_{\text{out}} \leftarrow \texttt{var}(\text{confs}_{\text{out}})$
14: $\text{conf}_{\text{obs}} = \phi(f(x)_y)$ ▷ *query target model*
15: **return** $\Lambda = \dfrac{p(\text{conf}_{\text{obs}} \mid \mathcal{N}(\mu_{\text{in}}, \sigma^2_{\text{in}}))}{p(\text{conf}_{\text{obs}} \mid \mathcal{N}(\mu_{\text{out}}, \sigma^2_{\text{out}}))}$

Carlini *et. al.*, https://arxiv.org/pdf/2112.03570

Estimating the likelihood-ratio with parametric modeling

- Please note that for any query data x, the attacker should train the corresponding shadow models!
  - Refer to this as an **online** attack
- Propose an **offline** version
  - For this attack, remove lines 5,6,10 and 12 from Algorithm 1, and only estimate the mean μ_out and variance σ_out of model confidences when the target example is not in the shadow models' training data.
- Change the likelihood-ratio test in line 15 to a one-sided hypothesis test. That is, measure the probability of observing a confidence as high as the target model's under the null-hypothesis that the target point (x, y) is a non-member:

$$\Lambda = 1 - \Pr[Z > \phi(f(x)_y)], \text{ where } Z \sim \mathcal{N}(\mu_{\text{out}}, \sigma^2_{\text{out}})$$

Algorithm 1 Our online Likelihood Ratio Attack (LiRA). We train shadow models on datasets with and without the target example, estimate mean and variance of the loss distributions, and compute a likelihood ratio test. (In our **offline** variant, we omit lines 5, 6, 10, and 12, and instead return the prediction by estimating a single-tailed distribution, as is shown in Equation (4).)

Require: model f, example (x, y), data distribution $\mathbb{D}$

1: $\text{confs}_{\text{in}} = \{\}$
2: $\text{confs}_{\text{out}} = \{\}$
3: **for** N times **do**
4: $\quad D_{\text{attack}} \leftarrow^{\$} \mathbb{D}$ ▷ *Sample a shadow dataset*
5: $\quad f_{\text{in}} \leftarrow \mathcal{T}(D_{\text{attack}} \cup \{(x, y)\})$ ▷ *train IN model*
6: $\quad \text{confs}_{\text{in}} \leftarrow \text{confs}_{\text{in}} \cup \{\phi(f_{\text{in}}(x)_y)\}$
7: $\quad f_{\text{out}} \leftarrow \mathcal{T}(D_{\text{attack}} \backslash \{(x, y)\})$ ▷ *train OUT model*
8: $\quad \text{confs}_{\text{out}} \leftarrow \text{confs}_{\text{out}} \cup \{\phi(f_{\text{out}}(x)_y)\}$
9: **end for**
10: $\mu_{\text{in}} \leftarrow \texttt{mean}(\text{confs}_{\text{in}})$
11: $\mu_{\text{out}} \leftarrow \texttt{mean}(\text{confs}_{\text{out}})$
12: $\sigma^2_{\text{in}} \leftarrow \texttt{var}(\text{confs}_{\text{in}})$
13: $\sigma^2_{\text{out}} \leftarrow \texttt{var}(\text{confs}_{\text{out}})$
14: $\text{conf}_{\text{obs}} = \phi(f(x)_y)$ ▷ *query target model*
15: **return** $\Lambda = \dfrac{p(\text{conf}_{\text{obs}} \mid \mathcal{N}(\mu_{\text{in}}, \sigma^2_{\text{in}}))}{p(\text{conf}_{\text{obs}} \mid \mathcal{N}(\mu_{\text{out}}, \sigma^2_{\text{out}}))}$

Carlini *et. al.*, https://arxiv.org/pdf/2112.03570

Experimental results

- Datasets:
  - CIFAR-10, CIFAR-100,ImageNet (a standard challenging image classification task)
  - WikiText-103 (a natural language processing text dataset).
- Target Models:
  - For CIFAR-100, follow the same process as for CIFAR-10 and train a wide ResNet to 60% accuracy on half of the dataset (25,000 examples).
  - For ImageNet, train a ResNet-50 on 50% of the dataset (roughly half a million examples).
  - For WikiText-103, use the GPT-2 tokenizer to split the dataset into a million sentences and train a small GPT-2 model on 50% of the dataset for 20 epochs to minimize the cross-entropy loss.
- Shadow models
  - For each dataset, the adversary trains N shadow models (N = 64 for ImageNet, and N = 256 otherwise)
  - Shadow training sets are chosen so that each example (x, y) is contained in exactly half of the shadow models' training sets (thus, for each example we have N/2 IN models, and N/2 OUT models).
    - In the paper some discussion on how to relax such assumption, omitted here

Carlini *et. al.*, https://arxiv.org/pdf/2112.03570

Experimental results

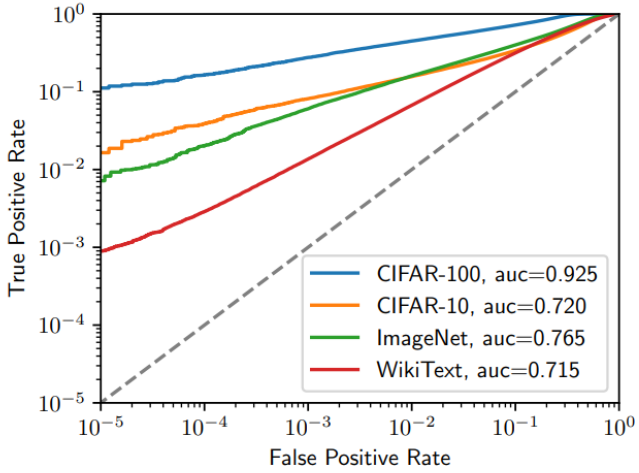


Fig. 5: **Success rate of our attack on CIFAR-10, CIFAR-100, ImageNet, and WikiText.** All plots are generated with 256 shadow models, except ImageNet which uses 64.

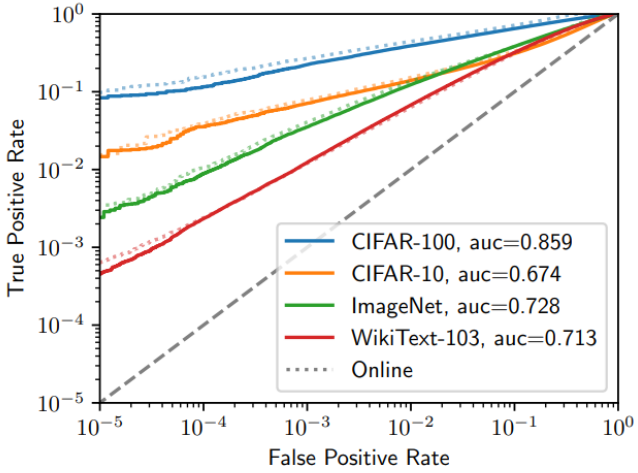


Fig. 6: **Success rate of our offline attack on CIFAR-10, CIFAR-100, ImageNet, and WikiText.** All plots are generated with 128 OUT shadow models, except ImageNet which uses 32. For each dataset, we also plot our online attack with the same number of shadow models (half IN, half OUT).

Attack has true-positive rates ranging from 0.1% to 10% at a false-positive rate of 0.001%.

Carlini *et. al.*, https://arxiv.org/pdf/2112.03570

Experimental results

Method	shadow models	multiple queries	class hardness	example hardness	TPR @ 0.001% FPR			TPR @ 0.1% FPR			Balanced Accuracy		
					C-10	C-100	WT103	C-10	C-100	WT103	C-10	C-100	WT103
Yeom et al. [70]	○	○	○	○	0.0%	0.0%	0.00%	0.0%	0.0%	0.1%	59.4%	78.0%	50.0%
Shokri et al. [60]	●	○	●	○	0.0%	0.0%	–	0.3%	1.6%	–	59.6%	74.5%	–
Jayaraman et al. [25]	○	●	○	○	0.0%	0.0%	–	0.0%	0.0%	–	59.4%	76.9%	–
Song and Mittal [61]	●	○	●	○	0.0%	0.0%	–	0.1%	1.4%	–	59.5%	77.3%	–
Sablayrolles et al. [56]	●	○	●	●	0.1%	0.8%	0.01%	1.7%	7.4%	1.0%	56.3%	69.1%	**65.7%**
Long et al. [37]	●	○	●	●	0.0%	0.0%	–	2.2%	4.7%	–	53.5%	54.5%	–
Watson et al. [68]	●	○	●	●	0.1%	0.9%	0.02%	1.3%	5.4%	1.1%	59.1%	70.1%	65.4%
Ye et al. [69]	●	○	●	●	-	-	-	-	-	-	60.3%	76.9%	65.5%
Ours	●	●	●	●	**2.2%**	**11.2%**	**0.09%**	**8.4%**	**27.6%**	**1.4%**	**63.8%**	**82.6%**	65.6%

TABLE I: **Comparison of prior membership inference attacks** under the same settings for well-generalizing models on CIFAR-10, CIFAR-100, and WikiText-103 using 256 shadow models. Accuracy is only presented for completeness; we do not believe this is a meaningful metric for evaluating membership inference attacks. Full ROC curves are presented in Appendix A.

Carlini *et. al.*, https://arxiv.org/pdf/2112.03570

Summary…

- Membership Inference Attacks (MIAs): Loss-based attack
  - A static, absolute threshold does not control for the intrinsic complexity of each utterance
- Stronger MIAs: Reference-based attacks (MIA)
  - We calibrate the threshold for each utterance
  - So instead of loss, we use likelihood ratio with respect to a reference model:

$$LR(s) = \frac{p(s; \theta_R)}{p(s; \theta)}$$

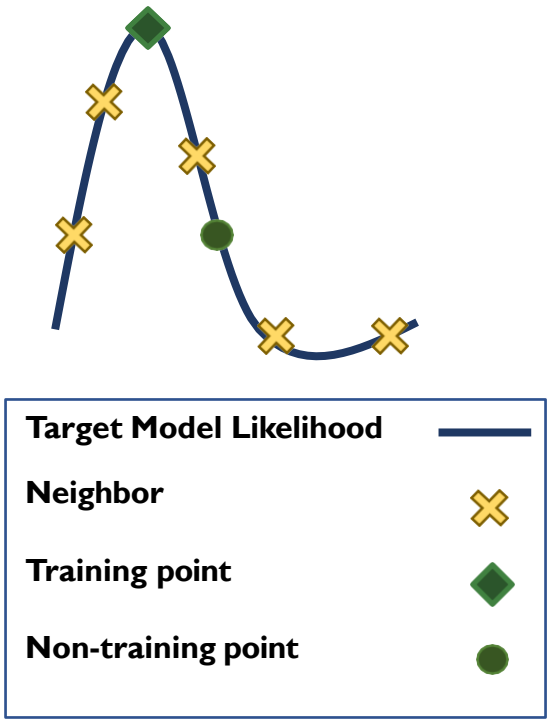
  - By thresholding the LR, we infer membership $if\ LR(s) < t \ \rightarrow s \in D$

… and Limitations

- The success of reference-based attacks is contingent upon having a good reference model, which is not always feasible:
  - We might have a very small dataset, therefore holding out part of the data to train a reference model on would significantly impact the utility of the final model
  - We might have limited/no information about the training data of the model we are probing, therefore curating non-overlapping, similar data would be non-trivial
  - We might not have access to enough compute to train large reference models

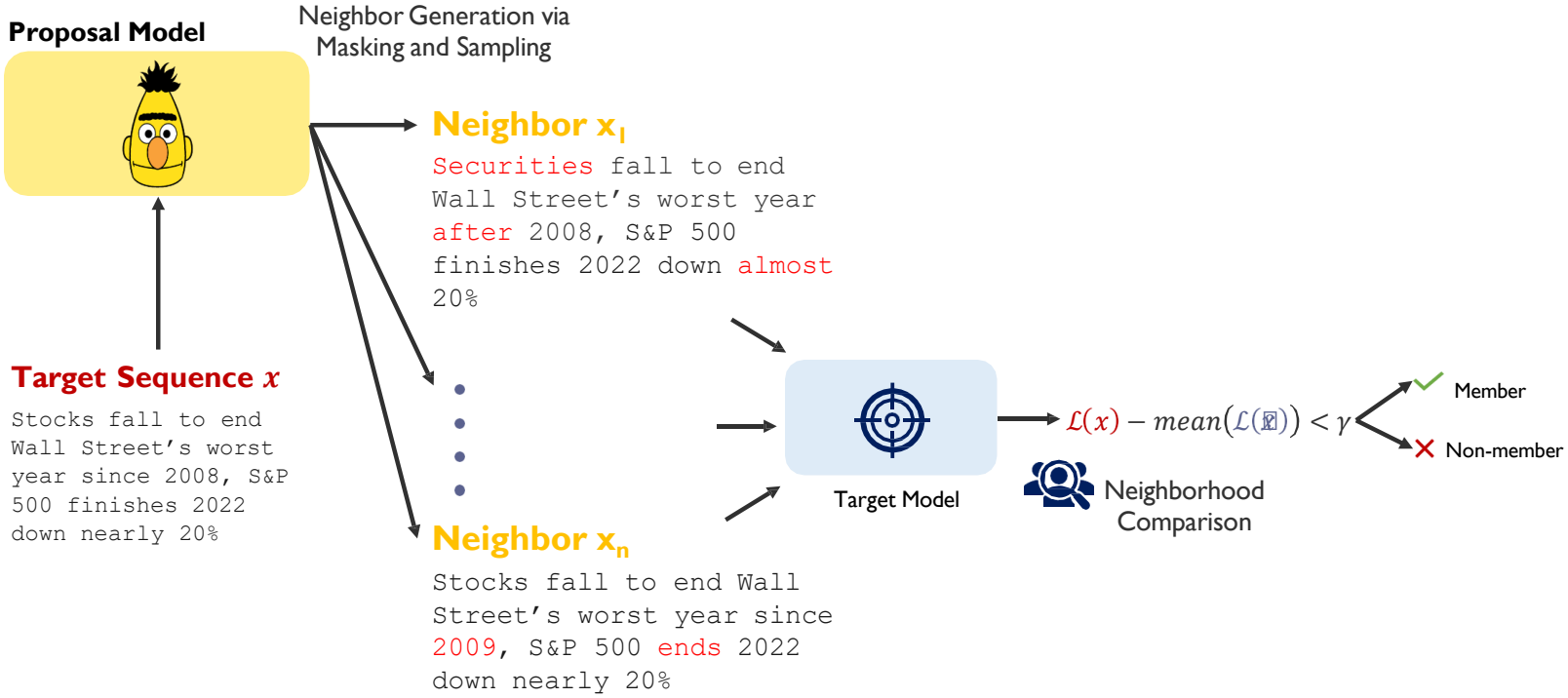
Neighborhood Comparison-based Attacks

- Instead of likelihood ratio, use local-optimality of each point as a signal to determine membership.
- The intuition is:
  - If a data point is part of the training-set, its likelihood would be locally optimal, compared to its **neighboring points**
  - If a data point is not part of the training set, then there would be points its neighborhood with both higher and lower likelihoods



Mattern et al., https://aclanthology.org/2023.findings-acl.719/

Attack Procedure



Mattern et al., https://aclanthology.org/2023.findings-acl.719/

Experimental Setup

- Membership inference attack on fine-tuned GPT2
  - Let’s discuss for a moment why this is so important
- Baseline: Likelihood-ratio based attack
  - Trained LiRA attacks with shadow (reference models) that does not share the same distribution of the target one
    - Realistic and more difficult setting
  - Base reference: Pre-trained, non-finetuned model
  - Candidate reference: fine-tuned GPT2, but on a dataset with small distribution shift
- Oracle reference: fine-tuned GPT2 on a dataset with the same distribution as target model

Mattern et al., https://aclanthology.org/2023.findings-acl.719/

Experimental Results

	News			Twitter			Wikipedia		
False Positive Rate	1%	0.1%	0.01%	1%	0.1%	0.01%	1%	0.1%	0.01%
Likelihood Ratio Attacks:									
Base Reference Model	4.24%	0.91%	0.16%	5.66%	0.98%	0.22%	1.21%	0.12%	0.01%
Candidate Reference Model 1	4.91%	0.95%	0.15%	6.49%	1.10%	0.24%			
Candidate Reference Model 2	4.76%	0.92%	0.15%	6.61%	1.19%	0.25%			
Oracle Reference Model*	18.90%	3.76%	0.16%	13.90%	1.59%	0.28%	11.70%	3.70%	0.12%
Reference-Free Attacks:									
LOSS Attack	3.50%	0.10%	0.01%	2.08%	0.11%	0.02%	1.06%	0.11%	0.01%
Neighbour Attack (Ours)	**8.29%**	**1.73%**	**0.29%**	**7.35%**	**1.43%**	**0.28%**	**2.32%**	**0.27%**	**0.10%**

Table 2: True positive rates of various attacks for low false positive rates of $1\%, 0.1\%$, and 0.01%. Candidate Reference Model 1 refers to reference models trained on data from other AG News categories and our Twitter mental health dataset, Candidate Reference Model 2 refers to reference models trained on NewsCatcher and the offensive tweet classification dataset. *As reference attacks trained on oracle datasets represent a rather unrealistic scenario with perfect assumptions, we compare our results with other baselines with more realistic assumptions when highlighting best results as bold.

Mattern et al., https://aclanthology.org/2023.findings-acl.719/

Final Remarks on MIA

- How about larger models?
  - Of course this is not the end of the story:
  - There are all kinds of paper similar to the last one discussed here trying to build robust MIA with no reference/shadow model
    - Of course, as models become larger and larger, this aspect becomes increasingly important

Final Remarks on MIA

- How about larger models?
  - Some papers claim that MIA do not work anymore on large models:
    - This is mostly due to apparent low loss on “test” data
    - Duan et al. (2024b) demonstrate empirically that a range of MIA is not reliable against LLMs.
      - According to their findings, training for a single epoch - as is becoming standard practice for LLMs- and increasing amounts of training data may negatively affect MIA accuracy.
      - They also notice that MIA become more difficult as the difference between data included in the training set and data not included becomes more subtle.
      - However, as training corpora become larger – their statistics are based on the Pile (Gao et al., 2020) – this similarity increases: for example, non-members Github samples share on average 76.9 % of their 7-grams with the Pile.
    - The combined effect of training techniques that diverge from the one defined for other models and the huge training sets may challenge MIA’s effectiveness against LLMs.

Final Remarks on MIA

- How about larger models?
  - Some papers claim that MIA **do** work on large models:
    - Especially for long contexts… this will come back *soon*
    - Puerto et al. (2025) verify the efficacy of multiple MIA attacks varying the length of the tested examples
      - They construct a new benchmarks that measure the MIA performances at a continuous scale of data samples, from sentences (n-grams) to a collection of documents (multiple chunks of tokens).
    - They argue that MIA still works on LLMs, but only when multiple documents are presented for testing.

- As every topic in this course, no definitive solution yet to be found!

Acknowledgments

- Slide Content
  - Some slides adapted from:
    - https://trustworthy-ml-course.github.io/ Lecture 5
  - All papers and sources cited in slide
  - Slides and video from https://aclanthology.org/2023.findings-acl.719.pdf